Programming Assignment: Age Classification

Deep Learning — Spring 2026

Age Classification on the given Dataset

Phase I Deadline: **06-03-2026 11:59 PM**
Phase II Deadline: **15-03-2026 11:59 PM**
Dataset: Dataset Link
Starter Code: https://github.com/Divyaanshmertia/Deep-Learning-2026-Assignment-1
Submission: Upload `roll_no.py` + `roll_no.pdf` +`roll_no.pth`

1 Overview

In this assignment, you will build an image classifier that predicts the **age** (young vs. old) of a person from a face photograph. You are given a training set of 18,332 face images from the given dataset, split evenly between two classes.

Your goal is simple: **achieve the highest classification accuracy on the hidden test set.**

Your model will be evaluated by the instructor on a hidden test set, and your accuracy will be publicly posted post phase I.

Why this matters

Real-world classifiers often look great on training data but fail in production. This assignment will push you to think beyond training accuracy.

2 Dataset

2.1 Description

The dataset consists of aligned, cropped face images at 256×256 resolution. Each image belongs to one of two classes:

Label	Class
0	Young
1	Old

2.2 What you receive

- **Training images** (18,332 total): organised in class sub-folders.
- **Validation images** (134 total)

- **Validation labels** (`valid_labels.csv`): ground-truth labels for the validation images, so you can compute your own validation accuracy locally.
- **Starter code**: a training notebook and model definition file.
- **Evaluation script**: to verify your submission format before submitting.

Directory structure

```
dataset/
  train/
    0/          # 9,166 images (class 0 - Young)
    1/          # 9,166 images (class 1 - Old)
  valid/        # 134 images (flat, no sub-folders)
  valid_labels.csv
```

2.3 Download

Download the dataset from the link at the top of this document. The validation images and labels are provided so you can evaluate locally. The **test set is hidden**-only the instructor has access.

3 Your Task

Train a deep neural network to classify face images as *young* or *old*.

You have considerable freedom in **how** you train your model. You may use foundation models as a *reference* (e.g., to extract embeddings or auxiliary features).

The only hard constraint is the model architecture backbone (see below).

Constraint: Architecture and Training

You **must** use the **ResNet-18** architecture as provided by `torch` (see the starter notebook). You may make architectural modifications, provided that you do **not** change the depth or the core ResNet-18 layers.
You **must train** the ResNet-18 model yourself. The use of a pretrained ResNet-18 (or any other pretrained backbone) for the final model is **not allowed**.
You may use pretrained foundation models *only as references* (e.g., for embeddings or guidance), and they must **not** be used as the primary model.
You are **not allowed** to train multiple models.

3.1 What you CAN change

- Learning rate, weight decay, batch size, number of epochs, optimizer.
- Loss function (any suitable loss function or create your own loss function).
- Training strategy . . .).
- You may use **any paper, blog, or online resource** for ideas.

3.2 What you CANNOT change

- You must train on the **provided training data only** (no external face datasets).
- You must **not** attempt to reverse-engineer, look up, or hard-code test labels.

4 Submission

4.1 What to submit

You must submit **exactly two files**, named with your roll number:

File	Description
`roll_no.py`	A Python file containing your model class definition. This is needed so `torch.load()` can reconstruct your model.
`roll_no.pdf`	A **single-page PDF** file containing the core idea you have used to solve the age classification problem.
`roll_no.pth`	Your trained model, saved with `torch.save(model, 'roll_no.pth')`.

Important: Save the full model, not just weights

```
# use lower-case for roll_no!
# CORRECT
torch.save(model, 'roll_no.pth')

# WRONG --- do NOT do this
torch.save(model.state_dict(), 'roll_no.pth')
```

4.2 The `roll_no.py` file

Your `.py` file must contain the class definition used by the saved model. When Python unpickles a `torch.save(model, ...)` file, it needs the original class to be importable.

Example (if you use the provided ResNet-18 starter):

```
import torch.nn as nn
from torchvision import models

def build_model(num_classes=2):
    model = models.resnet18(weights=None)
    model.fc = nn.Linear(512, num_classes)
    return model
```

If you define a **custom model(R18 Only)**, put the full class definition in this file:

```python
import torch.nn as nn

class MyAgeClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        # ... Resnet 18 layers plus your additions...

    def forward(self, x):
        # ... your modified resnet18 forward pass ...
        return x
```

4.3 Self-check before submitting

Use the provided evaluation script to verify your submission:

```
python evaluate_submission_student.py \
    --model_path roll_no.pth \
    --model_file roll_no.py \
    --data_dir dataset/ \
    --valid
```

If this runs without errors and prints an accuracy, your submission is correctly formatted.

5 Evaluation & Leaderboard

- **Metric:** Classification accuracy (%).
- Your model is evaluated on hidden test images held by the instructor.
- **Final ranking** is based on the **hidden test set** accuracy, revealed after the deadline.

Hint

Check train and valid images and see if there is some pattern; see the accuracy of train and valid set what do you observe? Is it supposed to happen?

6 Phase I: Model Development

Duration:	11 days from release
Deadline:	**06-03-2026 11:59 PM**
Deliverable:	`roll_no.py` + `roll_no.pth` + `roll_no.pdf`

In Phase I, your objective is to develop the best age classifier you can with all the exploration that you desire.

1. **Start** from the provided starter notebook and model definition.

2. **Train** your model on the training set. You may modify anything except the optimizer.
3. **Evaluate** locally using the validation set and the provided evaluation script.
4. Once you are satisfied with your approach, before final submission train the entire model again this time **include the valid data provided to you!**
5. Prepare a **one-page PDF report** explaining your approach. *Reports exceeding one page will not be evaluated.*
6. **Submit** the following files by the deadline:
   - `roll_no.py`
   - `roll_no.pth`
   - `roll_no.pdf` (one page)

Leader-board and best performing models will be released on 8th March.

7 Phase II: Analysis & Improvement

Duration:	7 days after Phase I results are released
Deadline:	**15-03-2026 11:59 PM**
Deliverables:	`roll_no.py` + `roll_no.pth` + `roll_no.pdf`

After Phase I closes, we will release:

- The **best-performing model** and its training code from Phase I.
- The **final leaderboard** with test-set accuracies.

Your task in Phase II is twofold:

1. **Improve:** Build a model that achieves *higher accuracy* than the released best model. You may use the released code as a starting point or develop an entirely new approach.
2. **Analyse:** Write a **short report (1 page, PDF)** addressing the following questions:
   - What techniques did you use to improve performance your model, and why do they help?
   - Compare your Phase I and Phase II approaches.

8 Getting Started

1. **Clone the starter repository:**

```
git clone https://github.com/Divyaanshmertia/Deep-Learning-2026-
    Assignment-1
```

2. **Download the dataset** from the link at the top and extract it:

```
# Place the dataset so the structure looks like:
# age-assignment/dataset/train/0/
# age-assignment/dataset/train/1/
# age-assignment/dataset/valid/
# age-assignment/dataset/valid_labels.csv
```

3. **Install dependencies:**

```
pip install torch torchvision pillow numpy
```

4. **Open the starter notebook** (`starter_notebook.ipynb`) and run it end-to-end. This trains a baseline ResNet-18 and saves the model.

5. **Verify your submission** using the evaluation script:

```
python evaluate_submission_student.py \
    --model_path roll_no.pth \
    --model_file roll_no.py \
    --data_dir dataset \
```

6. **Submit** `roll_no.py` + `roll_no.pth` + `roll_no.pdf` before the deadline.

7. **Check the leaderboard** periodically to see where you stand!

9 Resources

- **Starter Code:** https://github.com/Divyaanshmertia/Deep-Learning-2026-Assignment-1
- **Dataset:** Dataset Link
- **Leaderboard:** visible after Phase 1
- **PyTorch Documentation:** pytorch.org/docs
- **torchvision Models:** pytorch.org/vision/stable/models

10 Academic Integrity

- You may discuss high-level ideas with classmates, but all submitted code must be your own.
- You may use any LLM (ChatGPT, Copilot, Claude, etc.) to help write or debug code.
- You may reference any paper, blog post, or tutorial.
- **Do not** share your trained model files or copy another student's code.
- **Do not** attempt to access, reconstruct, or reverse-engineer the hidden test labels.

- Zero marks will be awarded in the following cases:
  - Failure to submit your work on time.
  - Error in your code, or if it doesn't work on our end, or if your model does not work on our end during evaluation.
  - Absence from the scheduled viva.
  - Plagiarism.

Good luck! Your training accuracy might surprise you—but so might the validation acc.